Predicting Software Defects from Development History

Student Research Proposal • Najmus Sakib Yeamin • Machine Learning • Software Engineering • Data Analysis

Proposed by	Najmus Sakib Yeamin
Project type	Student-led research / prototype study
Estimated duration	8–10 weeks

Project Overview

I want to explore whether development-history data can be used to identify software modules that are more likely to contain defects. The project will focus on building an interpretable prediction pipeline from publicly available software-engineering datasets and comparing simple models rather than trying to create a production-quality defect predictor.

Student perspective
I want to keep the first version realistic for a student team: a focused research question, a reproducible experiment, and a prototype that can be discussed with a researcher and improved through collaboration.

Research Problem

Software teams have limited time for testing every part of a large codebase equally. Historical development information such as code changes, file size, and modification frequency may contain signals about where defects are more likely to occur. A student-scale experiment can investigate whether these signals are useful and how reliable simple prediction models can be.

Research Objectives

- Prepare a public software-engineering dataset for analysis.
- Study relationships between development metrics and defect labels.
- Build interpretable baseline classification models.
- Compare model performance and investigate important features.
- Analyze where predictions fail and discuss how the model could support, rather than replace, testing.

Research Questions

RQ1. Which development metrics show the strongest relationship with defects?

RQ2. Can simple classifiers provide a useful baseline?

RQ3. Which features remain useful across different software modules or projects?

RQ4. What types of modules are difficult for the models to classify correctly?

Proposed Methodology

Step 1. Select a suitable public dataset containing software metrics and defect labels.

Step 2. Clean the data and document feature definitions.

Step 3. Perform exploratory analysis and establish a simple baseline.

Step 4. Train several classification models and compare precision, recall, F1-score, and ROC-AUC where appropriate.

Step 5. Use feature importance or a simple explainability method to interpret results.

Step 6. Perform error analysis and discuss threats to validity and generalization.

Data / Materials / Experimental Resources

Public software-engineering datasets, Python notebooks, versioned experiment files, and documentation of feature definitions.

Evaluation Plan

Models will be compared using stratified evaluation and multiple metrics. Particular attention will be given to class imbalance and whether a high accuracy score hides poor defect detection.

Expected Challenges

- Different projects may use different coding and development practices.
- Defect datasets may be imbalanced or noisy.
- A model trained on one project may not generalize to another.
- Feature definitions must remain understandable and reproducible.

Reproducibility and Documentation

I will keep the main assumptions, dataset versions, preprocessing decisions, experiment settings, and evaluation results documented. The aim is that another student can understand how the result was produced and repeat the core experiment without relying on undocumented manual steps.

Planned Mitigation

I will begin with a small baseline and a feasibility check. If the original scope becomes too large for the available time or resources, I will narrow the research question rather than adding unnecessary complexity. Limitations and failed experiments will be documented as part of the research process.

Expected Outcomes

- A reproducible software-defect prediction pipeline.
- Comparison of several interpretable ML baselines.
- Feature and error analysis.
- A discussion of generalization and practical use in software testing.

Potential Research Collaboration

I am looking for someone interested in machine learning or software engineering who can help with dataset selection, feature interpretation, model evaluation, or literature review.

Technical Stack

Python, pandas, NumPy, scikit-learn, matplotlib, Jupyter Notebook, Git.

Scope, Ethics, and Responsible Research

Only public datasets suitable for research will be used. The study will not evaluate individual programmers or make claims about developer quality; the unit of analysis will be software artifacts or modules.

Proposed Work Plan

Phase	Activities
1. Background study	Literature review, problem definition, feasibility check
2. Setup	Data/system preparation and baseline implementation
3. Experiments	Development, comparison, tuning, and documentation
4. Evaluation	Metrics, error analysis, limitations, reproducibility
5. Reporting	Results, discussion, documentation, and presentation

What I Am Looking for in a Collaborator

I am looking for someone who is genuinely interested in the research question and willing to contribute to discussion, experiment design, implementation, analysis, or writing. I would like to divide responsibilities according to each person's strengths and keep the work transparent and documented.

Possible Next Step

If a researcher is interested, I would first like to discuss feasibility, refine the research question, and agree on a realistic division of work. The proposal can then be updated into a more specific study plan with milestones.

Note: This is a proposed student project. It does not claim completed research or guaranteed results, and the scope can be refined through collaboration.